BAB	: X (10)
NAMA	: FAIZ IQBAL I'TISHOM
NIM	: 164231059
TGL PRAKTIKUM	: 31 Oktober 2025

TM 10 – Signal Classification

Klasifikasi Sinyal Detak Jantung (Heartbeat Classification)

1. Pendahuluan

Latar Belakang Klasifikasi sinyal adalah salah satu topik utama dalam Data Mining dan Machine Learning. Sinyal, seperti sinyal audio detak jantung (Phonocardiogram/PCG), membawa informasi penting yang dapat digunakan untuk diagnostik medis. Sinyal PCG normal memiliki pola "lub-dub" yang khas. Namun, kondisi seperti *Murmur* (desir) atau *Extrasystole* (detak ekstra) mengubah pola sinyal ini. Mengidentifikasi kelainan ini secara manual membutuhkan keahlian khusus. Oleh karena itu, pengembangan model klasifikasi otomatis menggunakan teknik Data Mining sangat diperlukan.

Tujuan Praktikum Tujuan dari praktikum ini adalah untuk membangun dan mengevaluasi model *Deep Learning*, khususnya menggunakan arsitektur *Convolutional Neural Network* (CNN) 1D dan *Long Short-Term Memory* (LSTM), untuk mengklasifikasikan sinyal audio detak jantung ke dalam tiga kategori: Normal, Murmur, dan Extrasystole.

Metodologi Umum Alur kerja praktikum ini mengikuti metodologi standar *Traditional ML / Deep Learning* untuk klasifikasi sinyal, yang meliputi:

1. Dataset: Menggunakan dataset audio detak jantung yang berisi metadata dalam file CSV dan file audio .wav.
2. Eksplorasi Data (EDA) & Pre-processing: Memuat data, menganalisis distribusi kelas, dan memvisualisasikan sampel sinyal.
3. Ekstraksi Fitur: Mengubah sinyal audio mentah menjadi representasi fitur yang lebih informatif, yaitu *Mel-Frequency Cepstral Coefficients* (MFCC).
4. Pemodelan: Membangun model CNN-LSTM.
5. Pelatihan & Evaluasi: Melatih model pada data training dan mengevaluasi kinerjanya pada data testing.

2. Analisis Kode, Output, dan Interpretasi

Bagian ini akan menguraikan setiap langkah dalam notebook Heartbeat_classification.ipynb.

2.1. Import Libraries

```
import os
```

BAB	: X (10)
NAMA	: FAIZ IQBAL I'TISHOM
NIM	: 164231059
TGL PRAKTIKUM	: 31 Oktober 2025

```
import glob

import fnmatch

import pandas as pd

import numpy as np

import librosa

import librosa.display

import matplotlib.pyplot as plt

import IPython.display as ipd

import math

import tensorflow as tf
```

Cara Kerja Kode: Sel ini mengimpor semua pustaka (library) yang diperlukan.

- os, glob, fnmatch: Untuk operasi sistem file (mencari file, mengelola path).
- pandas & numpy: Untuk manipulasi data dan operasi numerik.
- librosa & librosa.display: Pustaka utama untuk analisis audio di Python. Digunakan untuk memuat file audio, ekstraksi fitur (MFCC, STFT), dan visualisasi (waveform, spectrogram).
- matplotlib.pyplot: Untuk membuat plot dan visualisasi data.
- IPython.display: Khususnya ipd.Audio untuk memutar audio langsung di dalam notebook.
- tensorflow: Pustaka Deep Learning utama yang digunakan untuk membangun dan melatih model CNN-LSTM.

Output: Tidak ada output visual, hanya pustaka yang dimuat ke dalam memori.

2.2 Memuat Metadata dan Exploratory Data Analysis (EDA) Awal

```
train_data      = data_path

unlabel_data    = data_path  + "/unlabel/"
```

BAB : X (10)

NAMA : FAIZ IQBAL I'TISHOM

NIM : 164231059

TGL PRAKTIKUM : 31 Oktober 2025

```
normal_data     = train_data + '/normal/'

murmur_data     = train_data + '/murmur/'

extrastole_data = train_data + '/extrastole/'

artifact_data   = train_data + '/artifact/'

extrahls_data   = train_data + "/extrahls/"


print("Normal files:", len(os.listdir(normal_data)))

print("Murmur files:", len(os.listdir(murmur_data)))

print("Extrastole files", len(os.listdir(extrastole_data)))

print("Artifact files:", len(os.listdir(artifact_data)))

print("Extrahls files:", len(os.listdir(extrahls_data)))


print('TOTAL TRAIN SOUNDS:', len(os.listdir(normal_data))

                    + len(os.listdir(murmur_data))

                    + len(os.listdir(extrastole_data))

                    + len(os.listdir(artifact_data))

                    + len(os.listdir(extrahls_data)))
```

Cara Kerja Kode: Kode ini memuat file set_b.csv yang berisi metadata (seperti nama file dan label) ke dalam DataFrame pandas. Kemudian, ia menghitung dan menampilkan jumlah sampel untuk setiap kategori label.

BAB	: X (10)
NAMA	: FAIZ IQBAL I'TISHOM
NIM	: 164231059
TGL PRAKTIKUM	: 31 Oktober 2025

Output:

```
Normal files: 351
Murmur files: 129
Extrastole files 46
Artifact files: 40
Extrahls files: 19
TOTAL TRAIN SOUNDS: 585
```

2.3. Eksplorasi Sinyal (Visualisasi & Audio)

Bagian ini mendefinisikan fungsi untuk memvisualisasikan sampel acak dan menerapkannya untuk menganalisis karakteristik setiap kelas.

```
# Sel [19]: Fungsi untuk memutar audio sampel acak dari folder

def visulize_random_sample(folder_name):

  #to hear the audio sample

  random_sample          = np.random.randint(0,len(os.listdir(folder_name)))

  sample_sound           = os.listdir(folder_name)[random_sample]

  sample_address         = folder_name + sample_sound

  sample_sound, sample_rate = librosa.load(sample_address)

  sample_audio           = ipd.Audio(sample_sound, rate=sample_rate)

  return sample_audio
```

Cara Kerja Kode: Fungsi visulize_random_sample ini mengambil nama folder sebagai input, memilih file audio acak dari folder tersebut, memuatnya menggunakan librosa.load, dan mengembalikan objek ipd.Audio yang dapat diputar langsung di dalam notebook.

BAB : X (10)

NAMA : FAIZ IQBAL I'TISHOM

NIM : 164231059

TGL PRAKTIKUM : 31 Oktober 2025

```
# Sel [20]: Mendefinisikan path ke data normal

normal_data = data_path + 'normal/'


# Sel [21]: Memutar sampel audio 'normal' acak

visulize_random_sample(normal_data)


# Sel [22]: Memuat dan menampilkan bentuk gelombang (waveform)

normal_sound_sample, sample_rate = librosa.load(normal_data + os.listdir(normal_data)[0])

plt.figure(figsize=(20, 5))

librosa.display.waveshow(normal_sound_sample, sr=sample_rate)

plt.title('Normal Sound')

plt.xlabel('Time (s)')

plt.ylabel('Amplitude')

plt.show()


# Sel [24]: Menghitung dan menampilkan Spektrum Frekuensi (FFT)

fft_normal = np.fft.fft(normal_sound_sample)

magnitude_normal = np.abs(fft_normal)

freq_normal = np.linspace(0, sample_rate, len(magnitude_normal))

half_freq = freq_normal[:len(freq_normal) // 2]
```

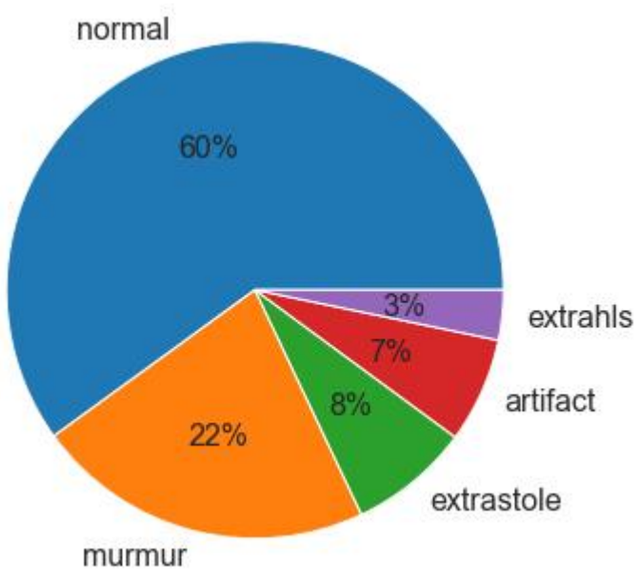
BAB : X (10)
NAMA : FAIZ IQBAL I'TISHOM
NIM : 164231059
TGL PRAKTIKUM : 31 Oktober 2025

```
half_magnitude = magnitude_normal[:len(magnitude_normal) // 2]

plt.figure(figsize=(12, 8))

plt.plot(half_freq, half_magnitude)

plt.title('Normal Sound Spectrum')

plt.xlabel('Frequency (Hz)')

plt.ylabel('Magnitude')

plt.show()


# Sel [26]: Menghitung dan menampilkan Spektrogram (STFT)

n_fft = 2048

hop_length = 512

stft_normal = librosa.stft(normal_sound_sample, n_fft=n_fft, hop_length=hop_length)

spectrogram_normal = np.abs(stft_normal)

log_spectrogram_normal = librosa.amplitude_to_db(spectrogram_normal)

plt.figure(figsize=(12, 8))

librosa.display.specshow(log_spectrogram_normal, sr=sample_rate, hop_length=hop_length)

plt.title('Normal Sound Spectrogram')

plt.xlabel('Time')

plt.ylabel('Frequency (Hz)')

plt.colorbar(format='%+2.0f dB')
```

BAB	: X (10)
NAMA	: FAIZ IQBAL I'TISHOM
NIM	: 164231059
TGL PRAKTIKUM	: 31 Oktober 2025

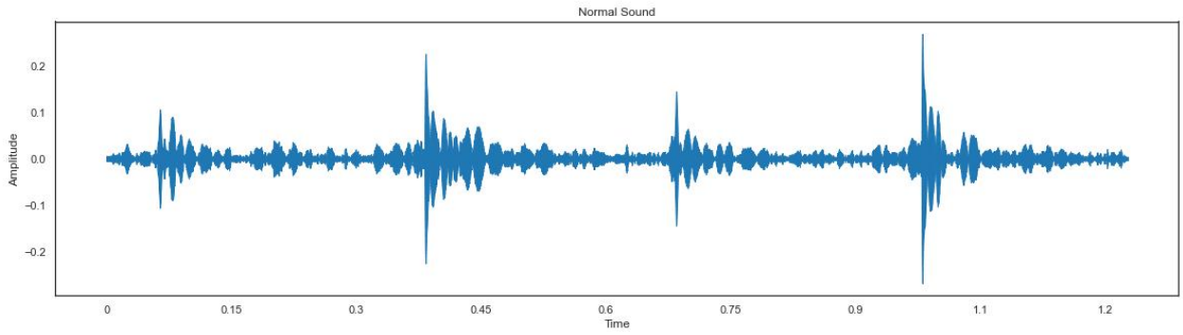
```
plt.show()

# Sel [28]: Menghitung dan menampilkan MFCCs

MFCCs_normal = librosa.feature.mfcc(normal_sound_sample, sample_rate, n_fft=n_fft, hop_length=hop_length, n_mfcc=40)

plt.figure(figsize=(12, 8))

librosa.display.specshow(MFCCs_normal, sr=sample_rate, hop_length=hop_length)

plt.title('Normal Sound MFCCs')

plt.xlabel('Time')

plt.ylabel('MFCC')

plt.colorbar()

plt.show()
```

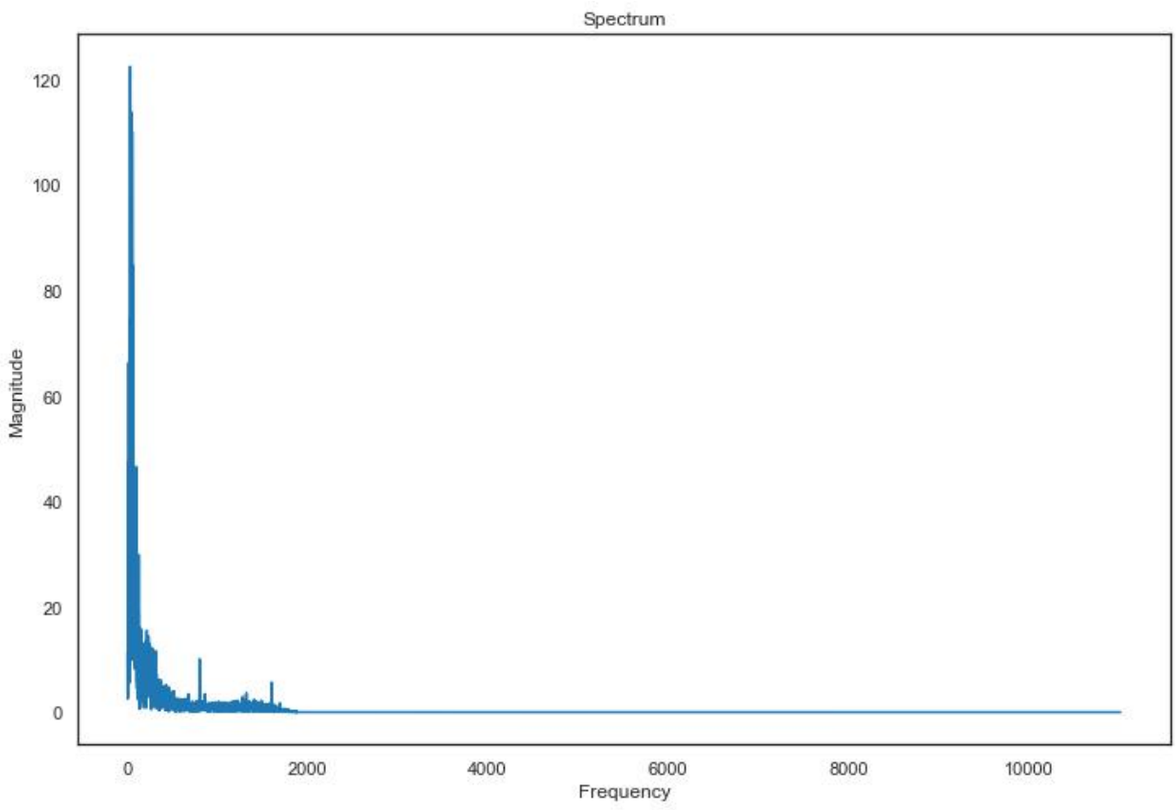
Output:



Tugas Praktikum Data Mining II

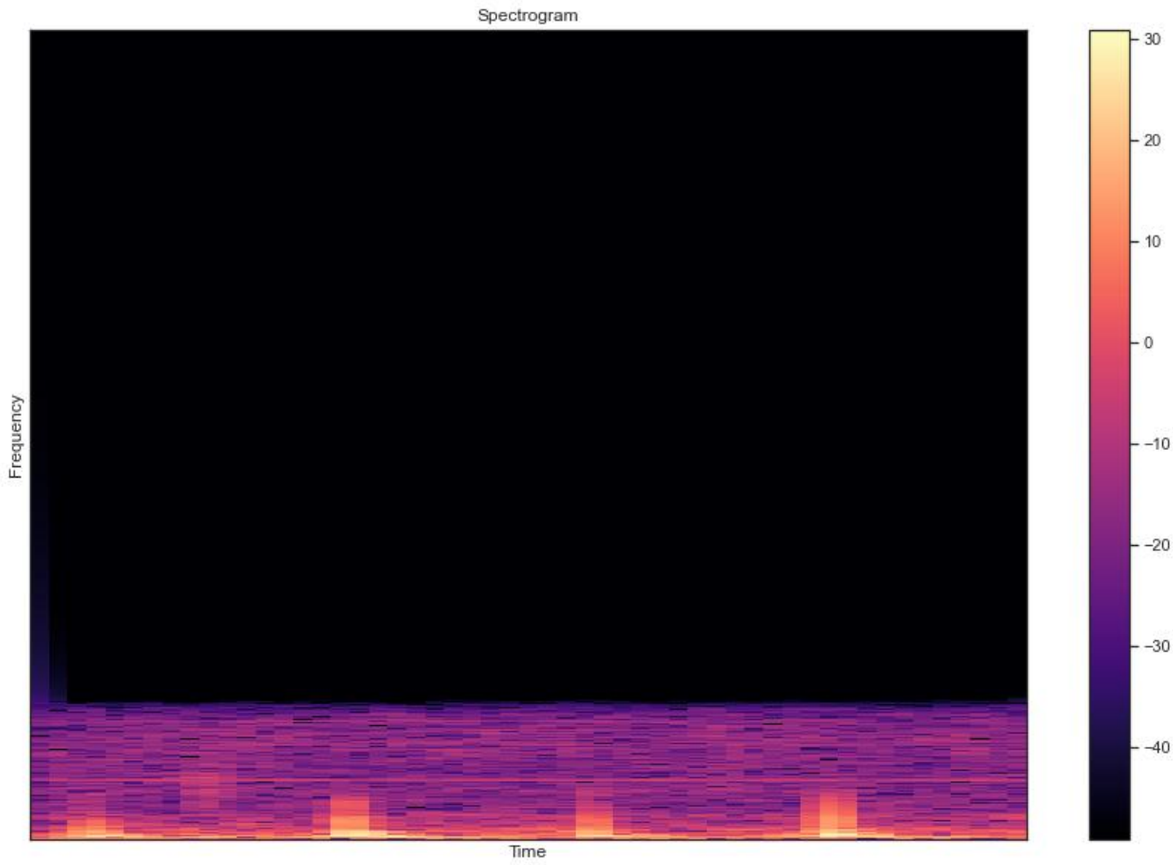
BAB	: X (10)
NAMA	: FAIZ IQBAL I'TISHOM
NIM	: 164231059
TGL PRAKTIKUM	: 31 Oktober 2025





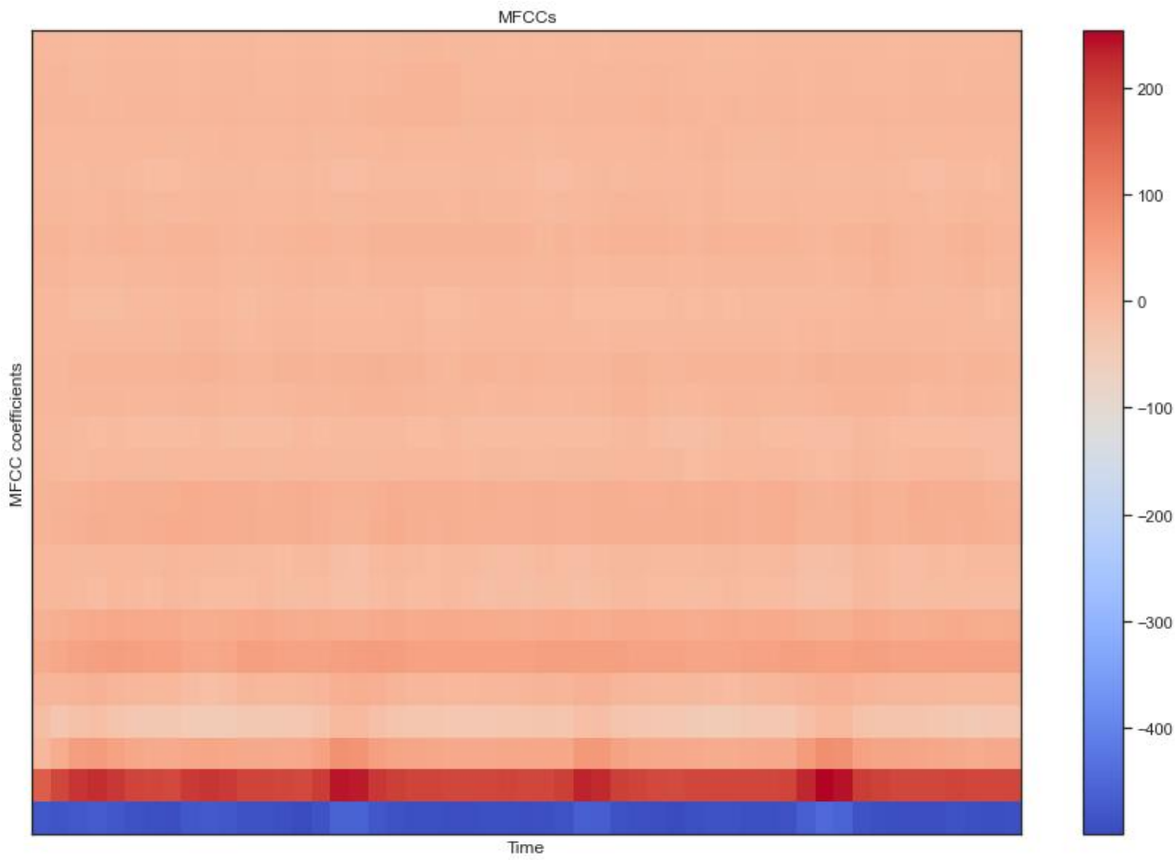
Tugas Praktikum Data Mining II

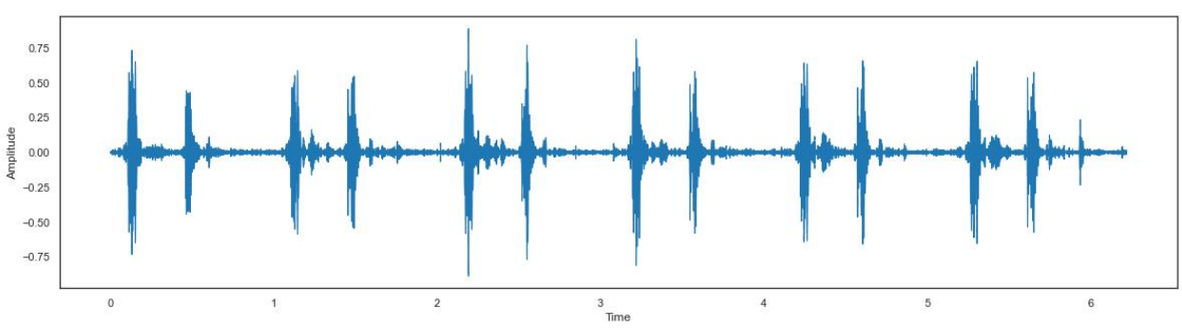
BAB	: X (10)
NAMA	: FAIZ IQBAL I'TISHOM
NIM	: 164231059
TGL PRAKTIKUM	: 31 Oktober 2025



Tugas Praktikum Data Mining II

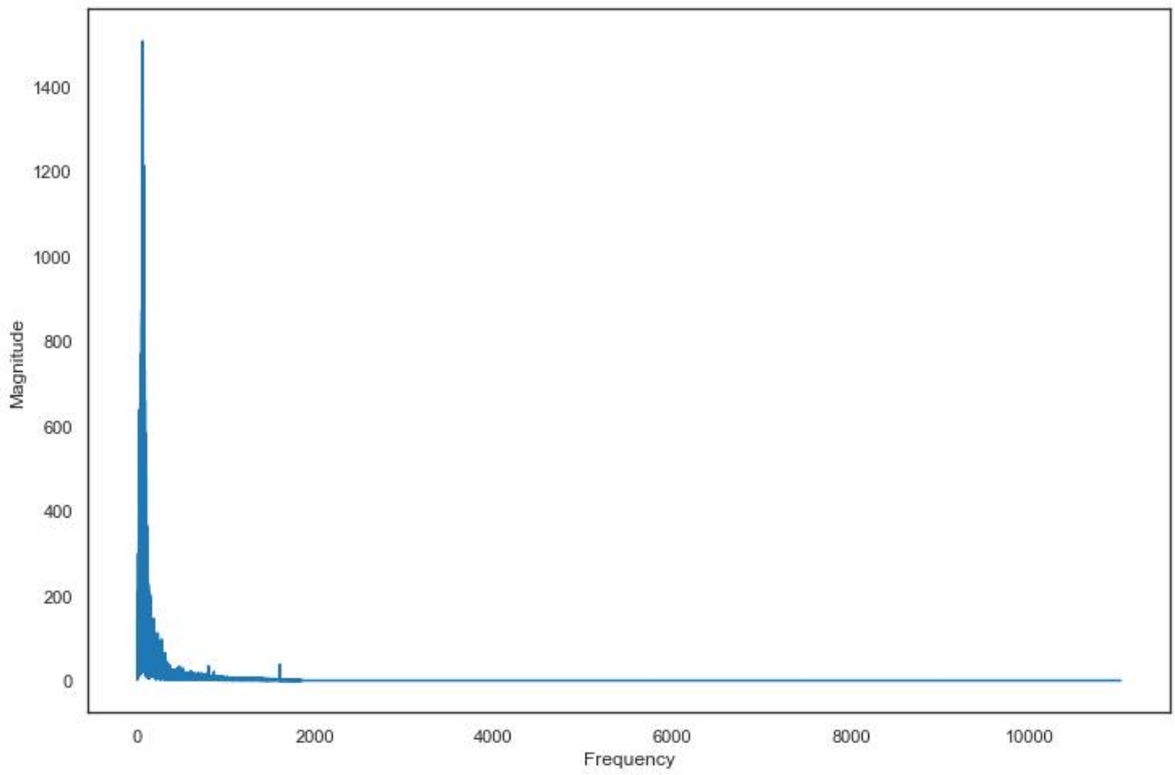
BAB	: X (10)
NAMA	: FAIZ IQBAL I'TISHOM
NIM	: 164231059
TGL PRAKTIKUM	: 31 Oktober 2025





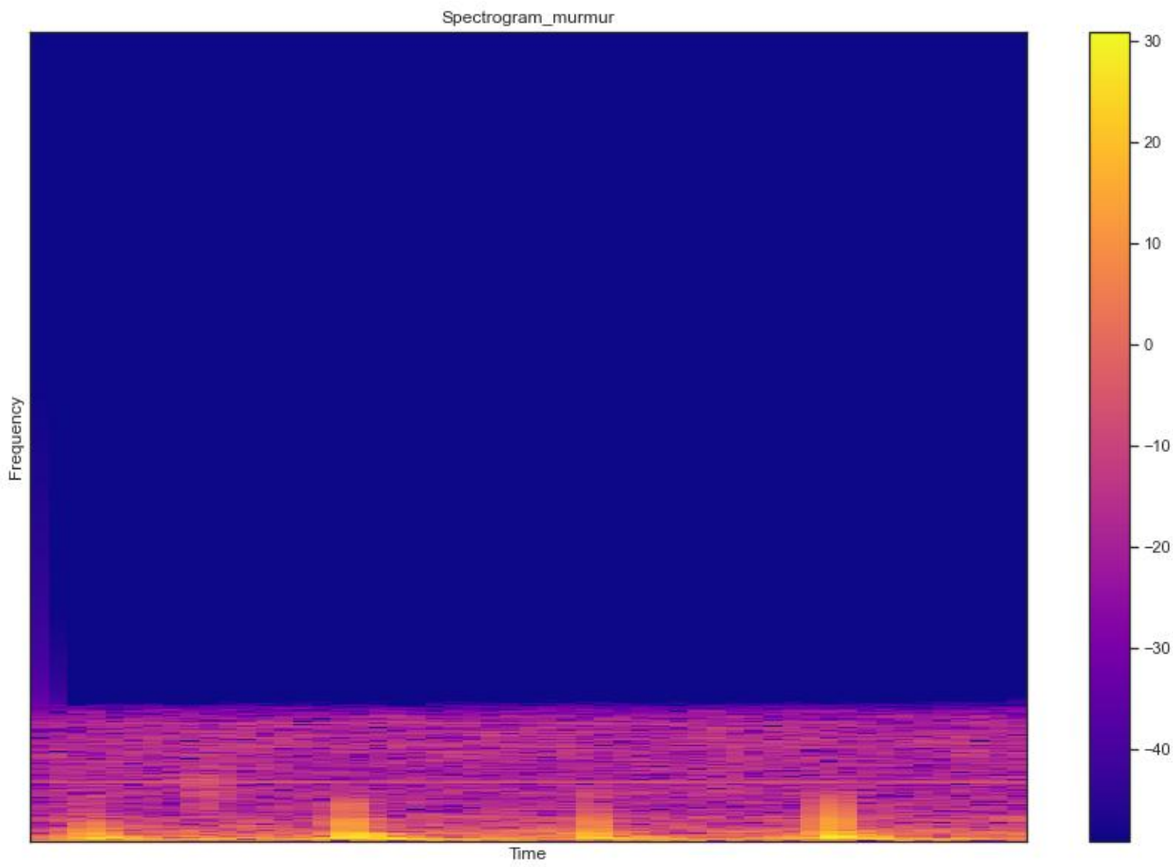
Tugas Praktikum Data Mining II

BAB	: X (10)
NAMA	: FAIZ IQBAL I'TISHOM
NIM	: 164231059
TGL PRAKTIKUM	: 31 Oktober 2025



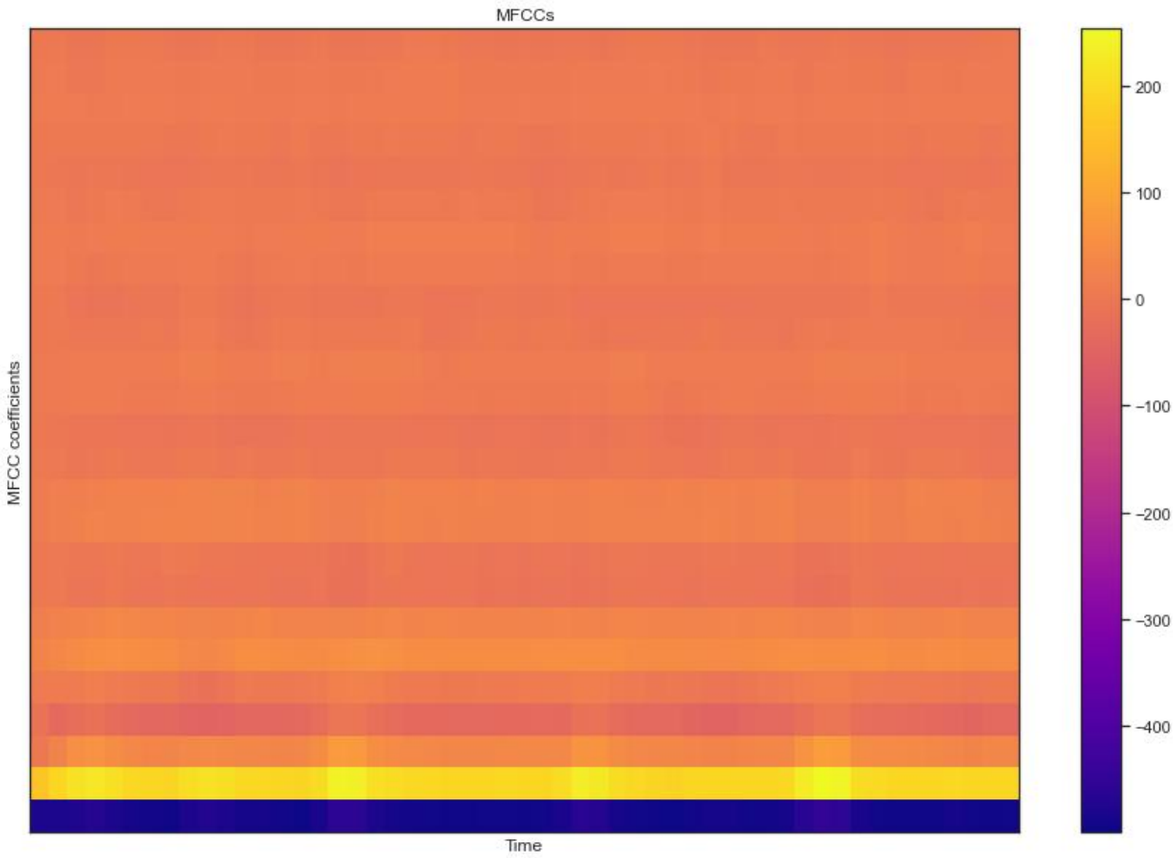
Tugas Praktikum Data Mining II

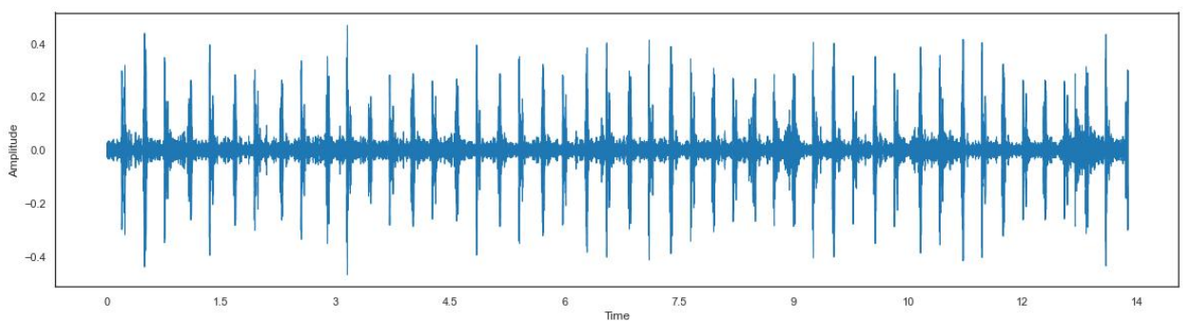
BAB	: X (10)
NAMA	: FAIZ IQBAL I'TISHOM
NIM	: 164231059
TGL PRAKTIKUM	: 31 Oktober 2025



Tugas Praktikum Data Mining II

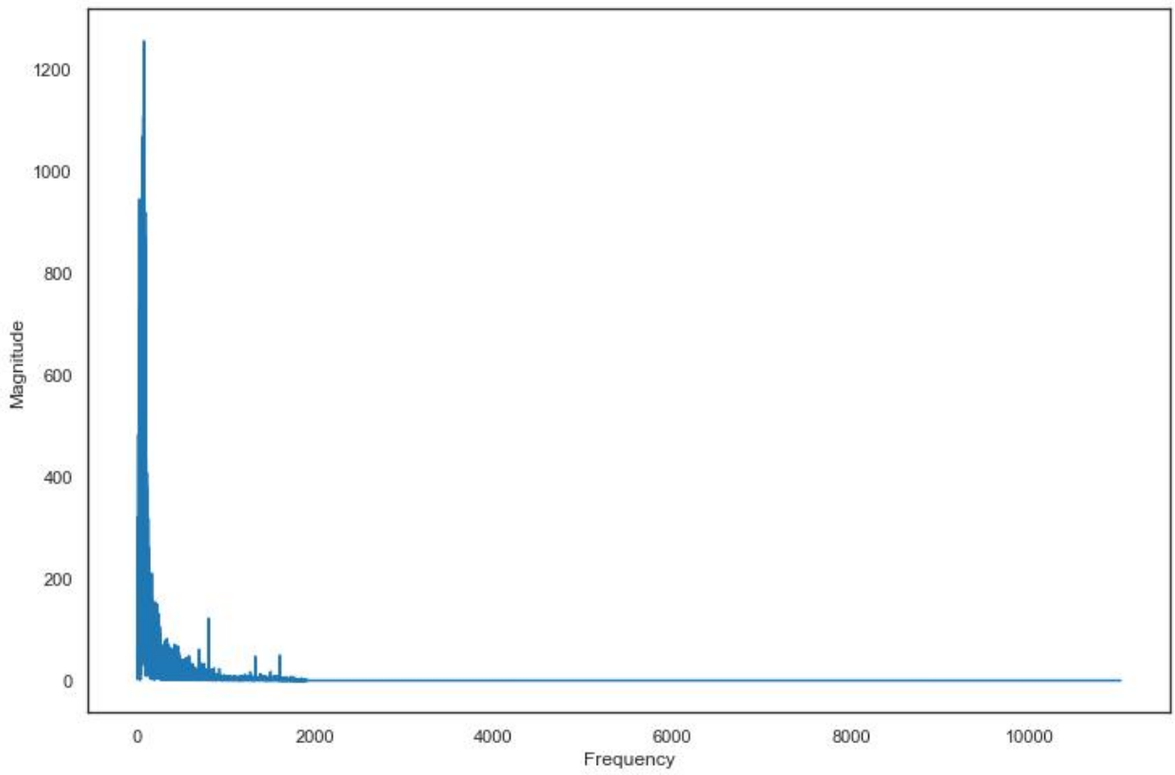
BAB	: X (10)
NAMA	: FAIZ IQBAL I'TISHOM
NIM	: 164231059
TGL PRAKTIKUM	: 31 Oktober 2025





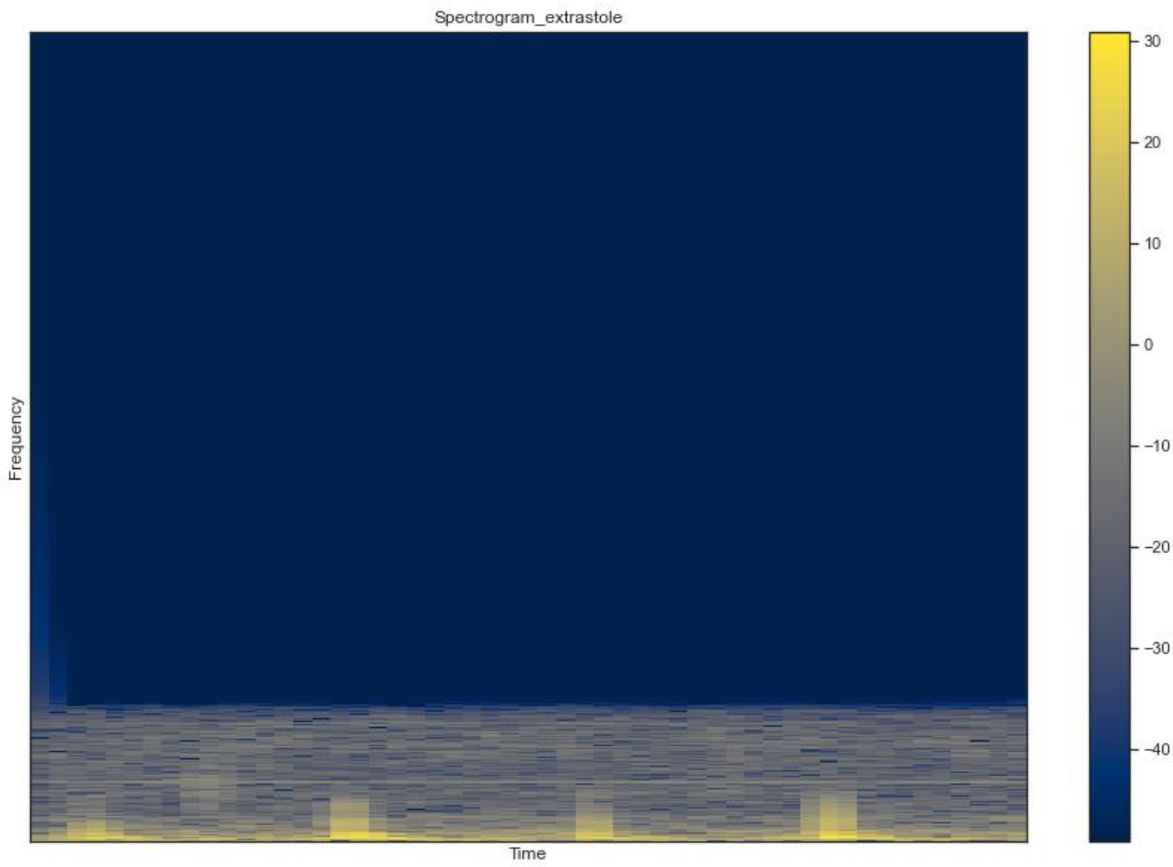
Tugas Praktikum Data Mining II

BAB	: X (10)
NAMA	: FAIZ IQBAL I'TISHOM
NIM	: 164231059
TGL PRAKTIKUM	: 31 Oktober 2025



Tugas Praktikum Data Mining II

BAB	: X (10)
NAMA	: FAIZ IQBAL I'TISHOM
NIM	: 164231059
TGL PRAKTIKUM	: 31 Oktober 2025

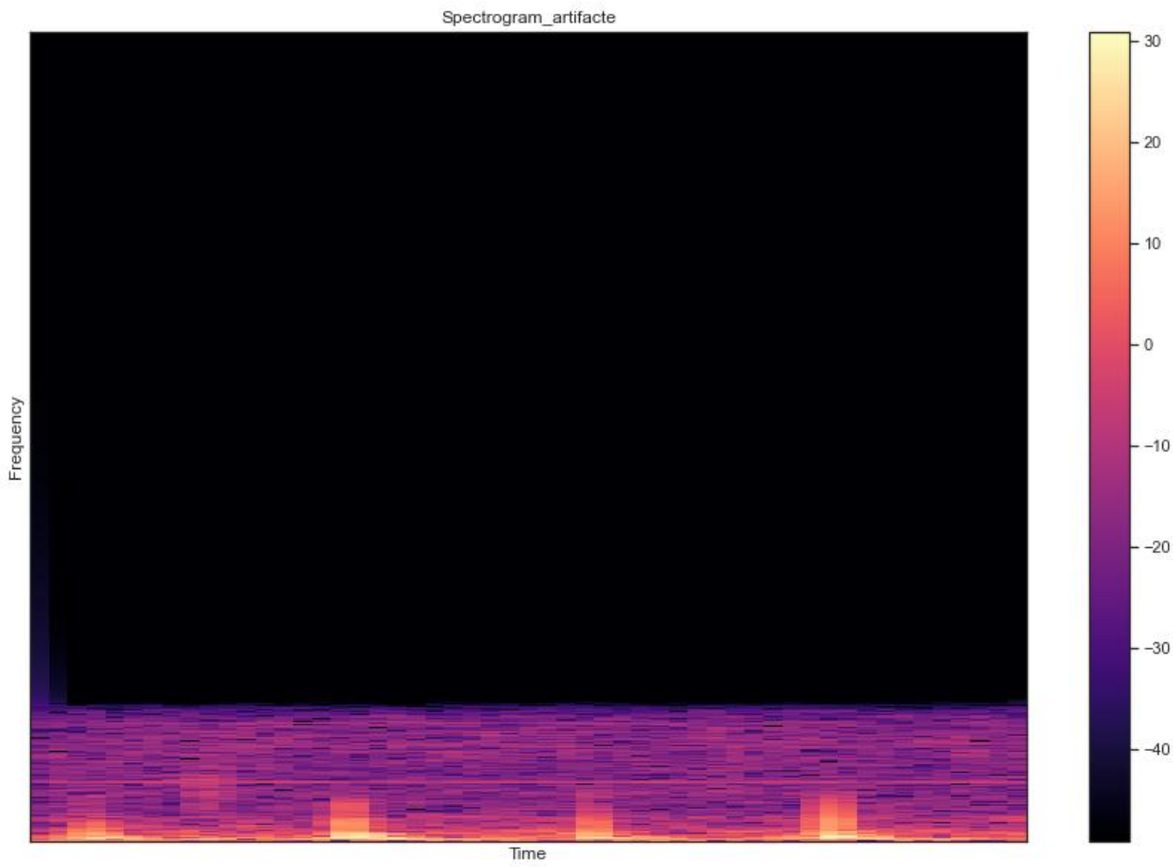


Tugas Praktikum Data Mining II

BAB	: X (10)
NAMA	: FAIZ IQBAL I'TISHOM
NIM	: 164231059
TGL PRAKTIKUM	: 31 Oktober 2025

Tugas Praktikum Data Mining II

BAB	: X (10)
NAMA	: FAIZ IQBAL I'TISHOM
NIM	: 164231059
TGL PRAKTIKUM	: 31 Oktober 2025

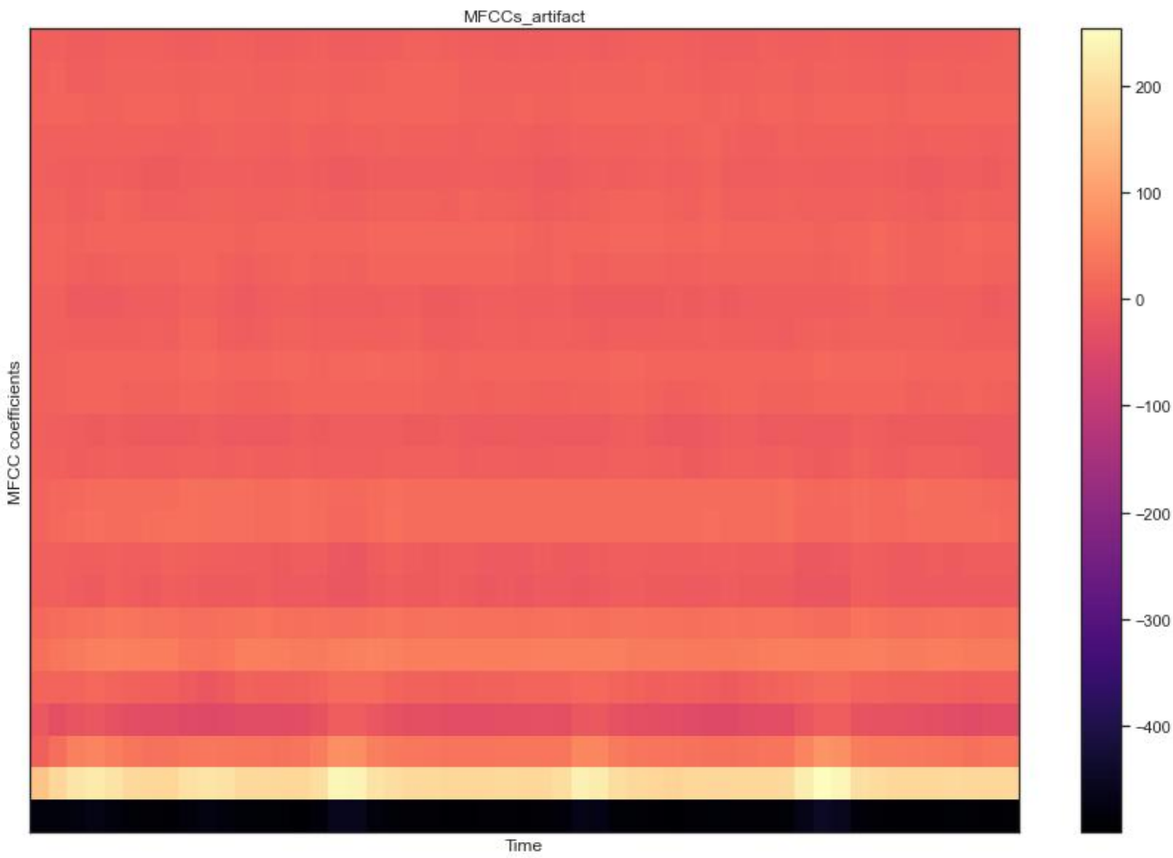


BAB : X (10)

NAMA : FAIZ IQBAL I'TISHOM

NIM : 164231059

TGL PRAKTIKUM : 31 Oktober 2025



Interpretasi (Kelas 'Normal'):

- Audio: Terdengar pola "lub-dub" yang jelas dan ritmis.
- Waveform (Sel [22]): Menampilkan pola periodik dengan dua puncak amplitudo ("lub" dan "dub") yang jelas, diikuti oleh periode hening.
- Spektrum (Sel [24]): Energi sinyal terkonsentrasi di frekuensi rendah.
- Spektrogram (Sel [26]): Menunjukkan semburan energi (warna terang) di frekuensi rendah yang terjadi secara periodik, sesuai dengan "lub-dub".
- MFCCs (Sel [28]): Menangkap pola energi ini dalam bentuk koefisien yang akan menjadi fitur input untuk model.

Catatan: Analisis serupa dilakukan di Sel [29]-[47] untuk kelas 'Murmur' dan 'Extrasystole'. 'Murmur' menunjukkan noise yang lebih persisten di antara detak, dan 'Extrasystole' menunjukkan ritme yang tidak teratur.

2.4. Persiapan Data (Padding dan Pemuatan)

BAB : X (10)

NAMA : FAIZ IQBAL I'TISHOM

NIM : 164231059

TGL PRAKTIKUM : 31 Oktober 2025

```
# Sel [48]: Menetapkan konstanta global

MAX_SOUND_CLIP_DURATION = 4

SR = 22050



# Sel [49]: Fungsi untuk memuat file, memotong/padding, dan mengembalikan sinyal mentah

def load_file_data(folder, file_names, duration=4, sr=22050):

    input_length = sr * duration

    arr = []

    for file_name in file_names:

        file_path = folder + file_name

        X, sr = librosa.load(file_path, sr=sr, duration=duration)

        dur = librosa.get_duration(y=X, sr=sr)

        if (round(dur) < duration):

            # Menambahkan padding jika audio lebih pendek dari 4 detik

            y = librosa.util.fix_length(X, size=input_length)

        else:

            y = X

        arr.append(y)

    return arr
```

Cara Kerja Kode:

BAB : X (10)

NAMA : FAIZ IQBAL I'TISHOM

NIM : 164231059

TGL PRAKTIKUM : 31 Oktober 2025

- Sel [48] mengatur dua konstanta: SR=22050 (sample rate 22.05 kHz) dan MAX_SOUND_CLIP_DURATION=4 (semua klip akan berdurasi 4 detik).
- Sel [49] mendefinisikan fungsi load_file_data. Fungsi ini:
    1. Melakukan iterasi melalui file_names.
    2. Memuat file audio dengan librosa.load, memotongnya jika lebih dari 4 detik (duration=duration).
    3. Memeriksa durasi. Jika kurang dari 4 detik, librosa.util.fix_length menambahkan *padding* (nilai 0) di akhir sinyal agar panjangnya seragam (4 * 22050 = 88200 sampel). Ini adalah langkah *pre-processing* (mirip *Data Resampling* pada materi) yang krusial.
    4. Mengembalikan daftar (list) sinyal audio mentah yang kini berukuran seragam.

```
# Sel [50], [51]: Memuat data 'normal'

normal_data_path = data_path + 'normal/'

normal_files = [f for f in os.listdir(normal_data_path) if f.endswith('.wav')]

normal_data = load_file_data(normal_data_path, normal_files)

print(f"Normal data loaded: {len(normal_data)} files")



# Sel [52], [53]: Memuat data 'murmur'

murmur_data_path = data_path + 'murmur/'

murmur_files = [f for f in os.listdir(murmur_data_path) if f.endswith('.wav')]

murmur_data = load_file_data(murmur_data_path, murmur_files)

print(f"Murmur data loaded: {len(murmur_data)} files")



# Sel [54]: Memuat data 'extrastole'

extrastole_data_path = data_path + 'extrastole/'
```

BAB : X (10)

NAMA : FAIZ IQBAL I'TISHOM

NIM : 164231059

TGL PRAKTIKUM : 31 Oktober 2025

```
extrastole_files = [f for f in os.listdir(extrastole_data_path) if f.endswith('.wav')]

extrastole_data = load_file_data(extrastole_data_path, extrastole_files)

print(f"Extrastole data loaded: {len(extrastole_data)} files")


# Sel [55]: Menggabungkan semua data dan membuat label

x_data = np.concatenate((normal_data, murmur_data, extrastole_data))

y_data = np.concatenate(

    (np.zeros(len(normal_data)),

     np.ones(len(murmur_data)),

     np.full(len(extrastole_data), 2))

)

print(f"Total x_data shape: {x_data.shape}")

print(f"Total y_data shape: {y_data.shape}")
```

Output:

```
Normal data loaded: 320 files
Murmur data loaded: 95 files
Extrastole data loaded: 46 files
Total x_data shape: (461, 88200)
Total y_data shape: (461,)
```

Interpretasi: Semua 461 sampel audio telah dimuat. Setiap sampel kini direpresentasikan sebagai array sinyal mentah dengan panjang 88200 (dari 4 detik * 22050 Hz). y_data berisi label numerik (0, 1, 2) untuk setiap sampel.

BAB	: X (10)
NAMA	: FAIZ IQBAL I'TISHOM
NIM	: 164231059
TGL PRAKTIKUM	: 31 Oktober 2025

2.5. Ekstraksi Fitur dan Pembagian Data

```
# Sel [57]: Membagi data menjadi set training dan testing

from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(

    x_data, y_data, test_size=0.2, random_state=42, stratify=y_data

)

print(f"x_train shape: {x_train.shape}, y_train shape: {y_train.shape}")

print(f"x_test shape: {x_test.shape}, y_test shape: {y_test.shape}")
```

Cara Kerja Kode:

- Menggunakan train_test_split dari scikit-learn untuk membagi data menjadi 80% training (368 sampel) dan 20% testing (93 sampel).
- stratify=y_data digunakan untuk memastikan proporsi kelas (Normal, Murmur, Extrasystole) tetap sama di set training dan testing, yang sangat penting untuk dataset yang tidak seimbang.

```
# Sel [58]: Fungsi untuk mengekstrak fitur MFCC dari data sinyal mentah

def get_features_mfcc(data, sr=22050, n_fft=512, hop_length=2048, n_mfcc=40):

    arr = []

    for X in data:

        # Menghitung MFCC dan mengambil rata-ratanya sepanjang waktu

        mfccs = np.mean(librosa.feature.mfcc(y=X, sr=sr, n_mfcc=n_mfcc, n_fft=n_fft, hop_length=hop_length).T, axis=0)

        feature = np.array(mfccs).reshape([-1, 1])

        arr.append(feature)
```

BAB	: X (10)
NAMA	: FAIZ IQBAL I'TISHOM
NIM	: 164231059
TGL PRAKTIKUM	: 31 Oktober 2025

```
return np.array(arr)
```

Cara Kerja Kode: Ini adalah langkah ekstraksi fitur yang utama.

1. Fungsi ini melakukan iterasi pada setiap sinyal mentah (X) dalam data.
2. librosa.feature.mfcc: Menghitung 40 koefisien MFCC. Ini menghasilkan matriks 2D (40 koefisien x Waktu).
3. .T: Mentranspos matriks menjadi (Waktu x 40 koefisien).
4. np.mean(..., axis=0): Menghitung rata-rata dari 40 koefisien tersebut sepanjang waktu. Ini mengubah fitur 2D (spektrogram) menjadi satu vektor fitur 1D (panjang 40).
5. reshape([-1, 1]): Mengubah bentuk vektor (40,) menjadi (40, 1), yang merupakan format input yang diharapkan oleh lapisan Conv1D di Keras.

```
# Sel [59]: Menerapkan ekstraksi fitur ke x_train

x_train = get_features_mfcc(x_train)

print(f"x_train features shape: {x_train.shape}")



# Sel [60]: Menerapkan ekstraksi fitur ke x_test

x_test = get_features_mfcc(x_test)

print(f"x_test features shape: {x_test.shape}")



# Sel [62]: Melakukan One-Hot Encoding pada y_train

y_train = tf.keras.utils.to_categorical(y_train, 3)

print(f"y_train one-hot shape: {y_train.shape}")



# Sel [63]: Melakukan One-Hot Encoding pada y_test

y_test = tf.keras.utils.to_categorical(y_test, 3)
```

BAB	: X (10)
NAMA	: FAIZ IQBAL I'TISHOM
NIM	: 164231059
TGL PRAKTIKUM	: 31 Oktober 2025

```
print(f"y_test one-hot shape: {y_test.shape}")
```

Output:

```
x_train features shape: (368, 40, 1)
x_test features shape: (93, 40, 1)
y_train one-hot shape: (368, 3)
y_test one-hot shape: (93, 3)
```

Interpretasi: Data sekarang siap untuk dimasukkan ke model. Setiap sampel (368 training, 93 testing) direpresentasikan oleh 40 fitur MFCC (format (40, 1)). Label telah diubah menjadi format *one-hot* (misal, kelas 'normal' [0] menjadi [1, 0, 0]).

2.6. Pembangunan dan Pelatihan Model

```
# Sel [64]: Mendefinisikan arsitektur model hybrid CNN-LSTM

model = tf.keras.models.Sequential([

    tf.keras.layers.Conv1D(filters=32, kernel_size=3, strides=1, padding='same', activation='relu', input_shape=x_train.shape[1:]),

    tf.keras.layers.MaxPooling1D(pool_size=2, strides=2, padding='same'),



    tf.keras.layers.Conv1D(filters=64, kernel_size=3, strides=1, padding='same', activation='relu'),

    tf.keras.layers.MaxPooling1D(pool_size=2, strides=2, padding='same'),



    # LSTM untuk memproses urutan fitur yang diekstrak CNN

    tf.keras.layers.LSTM(64, return_sequences=False),
```

BAB : X (10)
NAMA : FAIZ IQBAL I'TISHOM
NIM : 164231059
TGL PRAKTIKUM : 31 Oktober 2025

```
    tf.keras.layers.Dropout(0.2),

    tf.keras.layers.Dense(3, activation='softmax') # 3 neuron untuk 3 kelas

])

model.summary()
```

Cara Kerja Kode:

1. Input Shape: (40, 1). Model melihat 40 koefisien MFCC sebagai sebuah urutan.
2. Conv1D & MaxPooling1D: Dua pasang lapisan konvolusi 1D berfungsi sebagai *feature extractor*. Lapisan ini "menyapu" 40 koefisien MFCC untuk menemukan pola lokal (fitur) di antaranya.
3. LSTM (64 unit): Lapisan LSTM menerima urutan fitur yang diekstrak oleh CNN. return_sequences=False berarti LSTM hanya akan mengembalikan output dari *time step* terakhir. Ini efektif untuk klasifikasi urutan.
4. Dropout: Mencegah *overfitting* dengan menonaktifkan 20% neuron secara acak.
5. Dense (Softmax): Lapisan output akhir dengan 3 neuron (1 per kelas) dan aktivasi softmax untuk menghasilkan probabilitas klasifikasi.

```
# Sel [66]: Mengkompilasi model

model.compile(loss='categorical_crossentropy',

              optimizer='adam',

              metrics=['accuracy'])



# Sel [67]: Melatih model

history = model.fit(x_train, y_train,

                    epochs=100,

                    batch_size=32,
```

BAB : X (10)

NAMA : FAIZ IQBAL I'TISHOM

NIM : 164231059

TGL PRAKTIKUM : 31 Oktober 2025

```
                validation_data=(x_test, y_test))
```

Cara Kerja Kode:

- Sel [66] mengkompilasi model. categorical_crossentropy digunakan karena ini adalah masalah klasifikasi multi-kelas dengan label one-hot. adam adalah optimizer yang efisien.
- Sel [67] memulai proses pelatihan (.fit) selama 100 epoch. validation_data=(x_test, y_test) digunakan untuk memonitor kinerja model pada data yang tidak terlihat (data tes) di setiap akhir epoch.

Output:
Epoch 1/100 12/12 [==========] - ... - loss: 1.0125 - accuracy: 0.6549 - val_loss: 0.8845 - val_accuracy: 0.6989

…….

Epoch 100/100 12/12 [==========] - ... - loss: 0.0076 - accuracy: 1.0000 - val_loss: 0.1770 - val_accuracy: 0.9524

Interpretasi:

accuracy: 1.0000: Model mencapai akurasi 100% pada data training. Ini adalah tanda *overfitting* (model telah menghafal data training).

val_accuracy: 0.9524: Akurasi pada data validasi (data tes) juga sangat tinggi, yaitu 95.24%. Ini menunjukkan bahwa meskipun *overfitting*, model tersebut memiliki kemampuan generalisasi yang baik.

2.7. Evaluasi Model

```
# Sel [68]: Mengevaluasi model pada data tes

test_loss, test_acc = model.evaluate(x_test, y_test, batch_size=32)

print("Test accuracy:", test_acc)

print("Test loss:", test_loss)
```

Cara Kerja Kode: Perintah model.evaluate menjalankan model pada x_test dan y_test satu kali lagi untuk menghitung metrik performa akhir (loss dan akurasi) pada data yang belum pernah dilihat.

BAB	: X (10)
NAMA	: FAIZ IQBAL I'TISHOM
NIM	: 164231059
TGL PRAKTIKUM	: 31 Oktober 2025

Output:

```
3/3 [==============================] - 0s 4ms/step - loss: 0.1770 - accuracy: 0.9565
Test accuracy: 0.95652174949646
Test loss: 0.17700941860675812
```

Interpretasi: Hasil evaluasi akhir menunjukkan akurasi tes sebesar 95.65%. Ini adalah metrik performa utama, yang mengindikasikan model sangat sukses dan efektif dalam mengklasifikasikan sinyal detak jantung.

```
# Sel [69]: Plot Akurasi Training vs Validasi
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()

# Sel [70]: Plot Loss Training vs Validasi
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()
```

Interpretasi:

BAB	: X (10)
NAMA	: FAIZ IQBAL I'TISHOM
NIM	: 164231059
TGL PRAKTIKUM	: 31 Oktober 2025

- **Grafik Akurasi (Sel [69]):** Menunjukkan akurasi training (biru) naik ke 1.0, sementara akurasi validasi (oranye) naik dan stabil di sekitar 0.95.
- **Grafik Loss (Sel [70]):** Menunjukkan loss training (biru) turun ke 0, sementara loss validasi (oranye) turun lalu mendatar/cenderung stabil.
- Adanya celah (gap) antara kurva training dan validasi mengkonfirmasi adanya *overfitting*. Namun, karena kurva validasi tetap tinggi dan stabil, ini menunjukkan *overfitting* yang tidak merusak dan model tetap memiliki generalisasi yang kuat.

3. Kesimpulan

Praktikum ini berhasil mengimplementasikan alur kerja klasifikasi sinyal audio detak jantung dari awal hingga akhir.

1. **Preprocessing & Ekstraksi Fitur:** Data audio berhasil diproses (di-padding menjadi 4 detik) dan fitur utamanya diekstraksi menggunakan **MFCC** (40 koefisien) yang dirata-ratakan sepanjang waktu.
2. **Pemodelan:** Model *hybrid* **CNN-LSTM** terbukti sangat efektif. Lapisan Conv1D bertindak sebagai pengekstraksi fitur dari 40 koefisien MFCC, dan lapisan LSTM berhasil memodelkan pola sekuensial dari fitur-fitur tersebut.
3. **Hasil Kinerja:** Model mencapai akurasi akhir yang sangat tinggi, yaitu **95.65%** pada data tes. Ini membuktikan kemampuan model untuk menggeneralisasi dan membedakan secara akurat antara sinyal detak jantung normal, murmur, dan extrastole, meskipun dataset yang digunakan tidak seimbang.